

BÁO CÁO THỰC HÀNH

Thực hành Nhập môn Mạng máy tính

Lab 3: Mã hóa bất đối xứng RSA Hàm băm và Chữ ký số

GVTH: Nguyễn Ngọc Trường

Ngày báo cáo: 2/04/2026

Nhóm 2 – NT101.Q21.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
24520946	Bửu Nguyễn Phúc Vĩnh Lập	Task 2.3, Task 2.4	50%
24521958	Nguyễn Ngọc Tuyền	Task 2.1, Task 2.2	50%

A. Task 2.1

- Ý tưởng :

- + đầu vào có p, q, e
- + tính $n, \phi(n)$, d
- + tạo khóa công khai $PU = (e, n)$ và khóa PR = (d, n)
- + dùng khóa đó để
 - Mã hóa số
 - Giải mã số
 - Mã hóa chuỗi
 - Giải mã chuỗi / bản mã cho sẵn

Hình ảnh minh họa các hàm :

```
def tao_khoa(p, q, e):
    n = p * q
    phi_n = (p - 1) * (q - 1)
    if math.gcd(e, phi_n) != 1:
        raise ValueError("e không nguyên tố cùng nhau với phi(n)")
    d = pow(e, -1, phi_n)
    khoa_cong_khai = (e, n)
    khoa_rieng = (d, n)
    return khoa_cong_khai, khoa_rieng, phi_n

def ma_hoa_so_nguyen(ban_ro, khoa):
    so_mu, n = khoa
    if not (0 <= ban_ro < n):
        raise ValueError("Ban ro phải thỏa 0 <= ban_ro < n")
    return pow(ban_ro, so_mu, n)

def giai_ma_so_nguyen(ban_ma, khoa):
    so_mu, n = khoa
    return pow(ban_ma, so_mu, n)

# Mã hóa chuỗi theo từng block và xuất ra Base64
def ma_hoa_chuoi_sang_base64(chuoi, khoa):
    so_mu, n = khoa
    du_lieu = chuoi.encode("utf-8")
    kich_thuoc_block_ma = (n.bit_length() + 7) // 8
    kich_thuoc_block_ro = max(1, (n.bit_length() - 1) // 8)
    ket_qua = bytearray()
    for i in range(0, len(du_lieu), kich_thuoc_block_ro):
        block = du_lieu[i:i + kich_thuoc_block_ro]
        gia_tri_block = int.from_bytes(block, "big")
        if gia_tri_block >= n:
            raise ValueError("Block ban ro >= n, cần giảm kích thước block")
        ban_ma = pow(gia_tri_block, so_mu, n)
        ket_qua.extend(ban_ma.to_bytes(kich_thuoc_block_ma, "big"))
    return base64.b64encode(bytes(ket_qua)).decode()
```

```

def giai_ma_base64_sang_chuoi(base64_ban_ma, khoa, kich_thuoc_block_ro=None):
    so_mu, n = khoa
    kich_thuoc_block_ma = (n.bit_length() + 7) // 8
    if kich_thuoc_block_ro is None:
        kich_thuoc_block_ro = max(1, (n.bit_length() - 1) // 8)
    du_lieu_ma = base64.b64decode(base64_ban_ma)
    danh_sach_block = [
        du_lieu_ma[i:i + kich_thuoc_block_ma]
        for i in range(0, len(du_lieu_ma), kich_thuoc_block_ma)
    ]
    ket_qua = bytearray()
    for block in danh_sach_block:
        gia_tri_ma = int.from_bytes(block, "big")
        gia_tri_ro = pow(gia_tri_ma, so_mu, n)
        block_ro = gia_tri_ro.to_bytes((gia_tri_ro.bit_length() + 7) // 8 or 1, "big")
        ket_qua.extend(block_ro)
    return ket_qua.decode("utf-8", errors="replace")

```

- Câu 1 :

Mỗi bộ (p,q,e) :

Tính $n = p \cdot q$

Tính $\phi(n) = (p-1)(q-1)$

Kiểm tra $\gcd(e, \phi(n)) = 1$

Tìm $d = e^{-1} \bmod \phi(n)$

Tính ra $pu = (e, n)$

$Pr(d, n)$

Bộ 1 :

- $p_1 = 11, q_1 = 17, e_1 = 7$

- $n_1 = 11 \cdot 17 = 187$

- $\phi(n_1) = 10 \cdot 16 = 160$

- tìm d sao cho $7d \equiv 1 \pmod{160}$

$\Rightarrow d = 23$

Vậy:

- $PU_1 = (7, 187)$

- $PR_1 = (23, 187)$

Bộ 2:

- $PU_2 = (17, 13588476140342208394395166469647627226674348541791)$

- $PR_2 = (7993221259024828467291262184080358019185876599873, \text{cùng } n)$

hình ảnh minh họa bộ 2 :

```

# CAU 1 - BO 2
p2 = 20079993872842322116151219
q2 = 676717145751736242170789
e2 = 17
khoa_cong_khai_2, khoa_rieng_2, phi_n_2 = tao_khoa(p2, q2, e2)
print("Khoa cong khai 2 =", khoa_cong_khai_2)
print("Khoa rieng 2 =", khoa_rieng_2)

```

Bộ 3

Vì đề cho hệ 16 nên bạn đổi sang số nguyên trước:

- $p3 = \text{int}(\text{"F7E75FDC469067FFDC4E847C51F452DF"}, 16)$
- $q3 = \text{int}(\text{"E85CED54AF57E53E092113E62F436F4F"}, 16)$
- $e3 = \text{int}(\text{"0D88C3"}, 16)$

Kết quả:

- $n3 = \text{E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1}$
- $d3 = \text{3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB}$

hình ảnh minh họa bộ 3

```
# CAU 1 - BO 3
p3 = int("F7E75FDC469067FFDC4E847C51F452DF", 16)
q3 = int("E85CED54AF57E53E092113E62F436F4F", 16)
e3 = int("0D88C3", 16)
khoa_cong_khai_3, khoa_rieng_3, phi_n_3 = tao_khoa(p3, q3, e3)
print("Khoa cong khai 3 =", (hex(khoa_cong_khai_3[0]), hex(khoa_cong_khai_3[1])))
print("Khoa rieng 3      =", (hex(khoa_rieng_3[0]), hex(khoa_rieng_3[1])))
# CAU 2
```

câu 2: Mã hóa/giải mã $M = 5$ bằng bộ 1**a) Trường hợp bảo mật**

Dùng public key để mã hóa:

$$C = 5^7 \bmod 187 = 146$$

Dùng private key để giải mã:

$$M = 146^{23} \bmod 187 = 5$$

b) Trường hợp xác thực

Dùng private key để mã hóa/ký:

$$C = 5^{23} \bmod 187 = 180$$

Dùng public key để giải mã/xác minh:

$$M = 180^7 \bmod 187 = 5$$

Hình ảnh minh họa câu 1 + 2

```
# CAU 1 + CAU 2
p1, q1, e1 = 11, 17, 7
khoa_cong_khai_1, khoa_rheng_1, phi_n_1 = tao_khoa(p1, q1, e1)
print("Khoa cong khai 1 =", khoa_cong_khai_1)
print("Khoa rieng 1      =", khoa_rheng_1)
print("phi(n) 1         =", phi_n_1)
ban_ro = 5

# Truong hop bao mat (Confidentiality)
ban_ma_bao_mat = ma_hoa_so_nguyen(ban_ro, khoa_cong_khai_1)
ban_ro_sau_giai_ma_bao_mat = giai_ma_so_nguyen(ban_ma_bao_mat, khoa_rheng_1)
print("Bao mat:")
print(" Ban ma =", ban_ma_bao_mat)
print(" Giai ma =", ban_ro_sau_giai_ma_bao_mat)

# Truong hop xac thuc (Authentication)
ban_ma_xac_thuc = ma_hoa_so_nguyen(ban_ro, khoa_rheng_1)
ban_ro_sau_giai_ma_xac_thuc = giai_ma_so_nguyen(ban_ma_xac_thuc, khoa_cong_khai_1)
print("Xac thuc:")
print(" Ban ma =", ban_ma_xac_thuc)
print(" Giai ma =", ban_ro_sau_giai_ma_xac_thuc)
```

Câu 3:**Ý tưởng**

- đổi chuỗi sang bytes
- chia thành block sao cho giá trị block < n
- mã hóa từng block bằng RSA
- ghép lại thành bytes

hình ảnh minh họa câu 3 :

```
# CAU 3
thong_diep = "The University of Information Technology."
ban_ma_base64_bo_2 = ma_hoa_chuoi_sang_base64(thong_diep, khoa_cong_khai_2)
print("Ban ma Base64 voi bo 2 =", ban_ma_base64_bo_2)
thong_diep_giai_ma = giai_ma_base64_sang_chuoi(ban_ma_base64_bo_2, khoa_rheng_2)
print("Thong diep sau giai ma =", thong_diep_giai_ma)
```

Câu 4 :

- Bản mã 1 (Base64) được giải mã bằng khóa 2 cho ra thông điệp: “The University of Information Technology.”
- Bản mã 2 (HEX) được giải mã bằng khóa 1 cho ra thông điệp: “Hello”
- Bản mã 3 (Base64) được giải mã bằng khóa 1 cho ra thông điệp: “Vietnam National University - Ho Chi Minh City”
- Bản mã 4 (Binary) được giải mã bằng khóa 2 cho ra thông điệp: “Cryptography”

Hình ảnh minh họa

```
# CAU 4
def chuyen_ban_ma_ve_bytes(chuoi_ban_ma, dinh_dang):
    if dinh_dang == "base64":
        return base64.b64decode(chuoi_ban_ma)
    elif dinh_dang == "hex":
        return bytes.fromhex(chuoi_ban_ma)
    elif dinh_dang == "bin":
        so = int(chuoi_ban_ma, 2)
        do_dai_byte = (len(chuoi_ban_ma) + 7) // 8
        return so.to_bytes(do_dai_byte, "big")
    else:
        raise ValueError("Dinh dang khong hop le")

def giai_ma_theo_block(du_lieu_ma, khoa):
    so_mu, n = khoa
    kich_thuoc_block_ma = (n.bit_length() + 7) // 8

    if len(du_lieu_ma) % kich_thuoc_block_ma != 0:
        return None

    ket_qua = bytearray()

    for i in range(0, len(du_lieu_ma), kich_thuoc_block_ma):
        block_ma = du_lieu_ma[i:i + kich_thuoc_block_ma]
        gia_tri_ma = int.from_bytes(block_ma, "big")
        gia_tri_ro = pow(gia_tri_ma, so_mu, n)

        block_ro = gia_tri_ro.to_bytes((gia_tri_ro.bit_length() + 7) // 8 or 1, "big")
        ket_qua.extend(block_ro)

    return bytes(ket_qua)
```



```
!apt-get update
!apt-get install -y openssl vim-common diffutils coreutils build-essential wget
!mkdir -p /content/lab03/task22
!ls -R /content/lab03

... Hit:1 https://cli.github.com/packages stable InRelease
Hit:2 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
Hit:3 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:4 https://r2u.stat.illinois.edu/ubuntu jammy InRelease
Hit:5 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:7 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Hit:8 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:9 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' is not a valid source
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
build-essential is already the newest version (12.9ubuntu3).
diffutils is already the newest version (1:3.8-0ubuntu2).
coreutils is already the newest version (8.32-4.1ubuntu1.3).
openssl is already the newest version (3.0.2-0ubuntu1.21).
vim-common is already the newest version (2:8.2.3995-1ubuntu2.26).
wget is already the newest version (1.21.2-2ubuntu1.1).
0 upgraded, 0 newly installed, 0 to remove and 18 not upgraded.
/content/lab03:
task22 task23

/content/lab03/task22:
shattered-1.pdf shattered-2.pdf

/content/lab03/task23:
```

2.2.1

Tạo 2 file HEX :

Hình ảnh minh họa

```
[8] 0 giây
%%bash
cd /content/lab03/task22

cat > m1.hex << 'EOF'
d131dd02c5e6eec4693d9a0698aff95c2fcbab58712467eab4004583eb8fb7f8955ad340609f4b30283e488832571415a085125e8f7cdc99fd91dbdf286
EOF

cat > m2.hex << 'EOF'
d131dd02c5e6eec4693d9a0698aff95c2fcbab50712467eab4004583eb8fb7f8955ad340609f4b30283e4888325f1415a085125e8f7cdc99fd91dbd7286
EOF
```

- Đổi HEX sang binary

```
[9] 0 giây
%%bash
cd /content/lab03/task22
xxd -r -p m1.hex m1.bin
xxd -r -p m2.hex m2.bin
ls -l m1.bin m2.bin

... -rw-r--r-- 1 root root 128 Apr  2 01:55 m1.bin
-rw-r--r-- 1 root root 128 Apr  2 01:55 m2.bin
```

- Đếm số byte khác nhau và tính MD5:

Hình ảnh minh họa


```
[10]
0
giấy
import hashlib

with open("/content/lab03/task22/m1.bin", "rb") as f:
    b1 = f.read()

with open("/content/lab03/task22/m2.bin", "rb") as f:
    b2 = f.read()

print("Length m1:", len(b1))
print("Length m2:", len(b2))
print("Different bytes:", sum(x != y for x, y in zip(b1, b2)))
print("MD5(m1):", hashlib.md5(b1).hexdigest())
print("MD5(m2):", hashlib.md5(b2).hexdigest())

... Length m1: 128
Length m2: 128
Different bytes: 6
MD5(m1): 79054025255fb1a26e4bc422aef54eb4
MD5(m2): 79054025255fb1a26e4bc422aef54eb4
```

Nhận xét :

- Hai thông điệp có đầu và là 128 byte nhưng khác nhau : 6 byte
- Tuy nhiên sau khi tính giá trị băm MD5 thì kết quả thu được lại giống nhau : 79054025255fb1a26e4bc422aef54eb4
- Điều này cho thấy hai thông điệp khác nhau vẫn có thể tạo ra cùng 1 giá trị băm MD5 => xảy ra xung đột hàm băm. Từ đó cho thấy MD5 không đảm bảo tính kháng va chạm và không phù hợp cho các ứng dụng yêu cầu tính toàn vẹn và xác thực

2.2.2

- upload 2 file hello và erase

Hình ảnh minh họa :

```
from google.colab import files
uploaded = files.upload()

Choose Files 2 files
erase(n/a) - 4072 bytes, last modified: 4/2/2026 - 100% done
hello(n/a) - 4072 bytes, last modified: 4/2/2026 - 100% done
Saving erase to erase
Saving hello to hello

!wget -O /content/lab03/task22/shattered-1.pdf "https://marc-stevens.nl/research/shattered.io/static/shattered-1.pdf"
!wget -O /content/lab03/task22/shattered-2.pdf "https://marc-stevens.nl/research/shattered.io/static/shattered-2.pdf"
!ls -lh /content/lab03/task22

... --2026-04-02 01:42:15-- https://marc-stevens.nl/research/shattered.io/static/shattered-1.pdf
Resolving marc-stevens.nl (marc-stevens.nl)... 141.224.218.251
Connecting to marc-stevens.nl (marc-stevens.nl)|141.224.218.251|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 422435 (413K) [application/pdf]
Saving to: '/content/lab03/task22/shattered-1.pdf'

/content/lab03/task 100%[=====] 412.53K 886KB/s in 0.5s

2026-04-02 01:42:16 (886 KB/s) - '/content/lab03/task22/shattered-1.pdf' saved [422435/422435]

--2026-04-02 01:42:16-- https://marc-stevens.nl/research/shattered.io/static/shattered-2.pdf
Resolving marc-stevens.nl (marc-stevens.nl)... 141.224.218.251
Connecting to marc-stevens.nl (marc-stevens.nl)|141.224.218.251|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 422435 (413K) [application/pdf]
Saving to: '/content/lab03/task22/shattered-2.pdf'

/content/lab03/task 100%[=====] 412.53K 930KB/s in 0.4s
```

- Đưa file vào đúng thư mục để cấp quyền chạy

Hình ảnh minh họa

```

1] 0 giây
!mv hello erase /content/lab03/task22/ 2>/dev/null || true
!chmod +x /content/lab03/task22/hello /content/lab03/task22/erase 2>/dev/null || true
!ls -l /content/lab03/task22

... total 856
-rwxr-xr-x 1 root root 4072 Apr 2 01:41 erase
-rwxr-xr-x 1 root root 4072 Apr 2 01:41 hello
-rw-r--r-- 1 root root 128 Apr 2 01:55 m1.bin
-rw-r--r-- 1 root root 257 Apr 2 01:55 m1.hex
-rw-r--r-- 1 root root 128 Apr 2 01:55 m2.bin
-rw-r--r-- 1 root root 257 Apr 2 01:55 m2.hex
-rw-r--r-- 1 root root 422435 Feb 22 2017 shattered-1.pdf
-rw-r--r-- 1 root root 422435 Feb 22 2017 shattered-2.pdf

```

- Chạy 2 chương trình , tính sMD5 , rồi so sánh nội dung của chúng

Hình ảnh minh họa

```

12] 0 giây
%%bash
cd /content/lab03/task22
./hello || true
./erase || true
md5sum hello erase
cmp -l hello erase | head
file hello erase

... da5c61e1edc0f18337e46418e48c1290 hello
da5c61e1edc0f18337e46418e48c1290 erase
1876 341 141
1902 263 63
1916 103 303
1940 27 227
1966 140 340
1980 215 15
hello: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, for GNU
erase: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, for GNU
bash: line 2: ./hello: No such file or directory
bash: line 3: ./erase: No such file or directory

```

Nhận xét :

Tiến hành chạy 2 chương trình thử thi từ console sau đó rính giá trị băm MD5 của từng file và so sánh nội dung nhị phân của chúng. Kết quả cho thấy hai file có cùng giá trị MD5 , nhưng phép so sánh byte bằng lệnh cmp -l cho thấy chúng khác nhau ở nhiều vị trí. Điều này chứng minh rằng MD5 không còn đảm bảo tính kháng va chạm và không còn phù hợp để dùng trong kiểm tra tính toàn vẹn hay xác thực phần mềm

2.2.3

- upload 2 file PDF

```

[14] 17 giây
from google.colab import files
uploaded = files.upload()

... Choose Files 2 files
shattered-2.pdf(application/pdf) - 422435 bytes, last modified: 4/2/2026 - 100% done
shattered-1.pdf(application/pdf) - 422435 bytes, last modified: 4/2/2026 - 100% done
Saving shattered-2.pdf to shattered-2.pdf
Saving shattered-1.pdf to shattered-1.pdf

```

- Tính SHA-1

Hình minh họa

```

%%bash
cd /content/lab03/task22
sha1sum shattered-1.pdf shattered-2.pdf
sha256sum shattered-1.pdf shattered-2.pdf
cmp -l shattered-1.pdf shattered-2.pdf | head

38762cf7f55934b34d179ae6a4c80cadccbb7f0a  shattered-1.pdf
38762cf7f55934b34d179ae6a4c80cadccbb7f0a  shattered-2.pdf
2bb787a73e37352f92383abe7e2902936d1059ad9f1ba6daaa9c1e58ee6970d0  shattered-1.pdf
d4488775d29bdef7993367d541064dbdda50d383f89f0aa13a6ff2e0894ba5ff  shattered-2.pdf
193 163 177
196 221 223
197 146 246
200 21 1
201 217 73
204 266 252
205 41 35
208 17 13
209 371 105
212 314 326

```

Nhận xét :

Hai tệp PDF khác nhau nhưng cho cùng 1 giá trị SHA-1 , chứng minh SHA-1 cũng không an toàn trước tấn công va chạm. Vì vậy SHA-1 không phù hợp với các ứng dụng yêu cầu tính toàn vẹn và độ xác thực cao

2.2.4**Nhận xét :**

Qua 3 trường hợp trên, ta thấy cả MD5 và SHA-1 đều mất tính kháng va chạm. Hai dữ liệu khác nhau vẫn có thể cho ra cùng 1 giá trị băm, điều này làm cho việc dùng MD5 hoặc SHA-1 trong kiểm tra toàn vẹn dữ liệu, xác thực file, chữ ký số hoặc phân phối phần mềm trở nên không còn đáng tin cậy. Trong thực tế nên sử dụng các hàm băm mạnh hơn như SHA-256 hoặc SHA-3

C. Task 2.3

Môi trường thực thi: google colab

Câu hỏi 1: Nếu độ dài của tệp tiền tố của bạn không phải là bội số của 64, điều gì sẽ xảy ra?

1.Cấu hình

```

!git clone https://github.com/zhiijieshi/md5collgen.git

%cd md5collgen

!make

!ls -l md5collgen

```

Thực thi pull trường trình md5collgen từ github về bằng lệnh để tạo môi trường.

2. Thực hiện

```
!echo -n "24520946" > prefix_not64.txt
!./md5collgen -p prefix_not64.txt -o out1.bin out2.bin > /dev/null
!echo "File 1 trước khi băm"
!xxd out1.bin
!echo -e "\n"
!echo "File 2 trước khi băm"
!xxd out2.bin
!echo -e "\n"
!echo "Khi băm cả 2 file"
!md5sum out1.bin out2.bin
```

Dùng prefix là mssv sau khi chạy chương trình in file, rồi sau đó băm so sánh

3. Kết quả

File 1 trước khi băm

```
00000000: 3234 3532 3039 3436 0000 0000 0000 0000 24520946.....
00000010: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000030: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000040: 6ae0 14df 7897 42e7 680e c5db bcb0 f528 j...x.B.h.....(
00000050: b4b8 aa05 8d41 6f30 13a6 7f46 b216 a30b ....Ao0...F....
00000060: ebec bae5 6ec8 eda3 0bea f231 d4a7 d41a ....n.....1....
00000070: 1933 bf6b 8708 2c96 7890 496c 7fe3 f4af .3.k.,.x.Il....
00000080: cbf3 4fa2 1b68 2b92 f17d 6471 138a 4a03 ..0..h+..}dq..J.
00000090: a4b6 2213 eaf3 e058 c117 f8c4 d118 5db2 .."....X.....].
000000a0: fb1e 1ffb 3939 8f26 c784 c0ef 96b5 73e2 ....99.&.....s.
000000b0: 817c b495 0f30 26e0 8560 2778 3f51 8c05 .|...0&..`'x?Q..
```

File 2 trước khi băm

```
00000000: 3234 3532 3039 3436 0000 0000 0000 0000 24520946.....
00000010: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000030: 0000 0000 0000 0000 0000 0000 0000 0000 .....
00000040: 6ae0 14df 7897 42e7 680e c5db bcb0 f528 j...x.B.h.....(
00000050: b4b8 aa85 8d41 6f30 13a6 7f46 b216 a30b ....Ao0...F....
00000060: ebec bae5 6ec8 eda3 0bea f231 d427 d51a ....n.....1..'..
00000070: 1933 bf6b 8708 2c96 7890 49ec 7fe3 f4af .3.k.,.x.I.....
00000080: cbf3 4fa2 1b68 2b92 f17d 6471 138a 4a03 ..0..h+..}dq..J.
00000090: a4b6 2293 eaf3 e058 c117 f8c4 d118 5db2 .."....X.....].
000000a0: fb1e 1ffb 3939 8f26 c784 c0ef 9635 73e2 ....99.&.....5s.
000000b0: 817c b495 0f30 26e0 8560 27f8 3f51 8c05 .|...0&..`'.'?Q..
```

Nhận xét:

Như vậy ta thấy chương trình đã tạo ra 2 chuỗi đúng như hình minh họa

-Phần prefix (mssv: 24520946) **giống nhau**

-Phần padding (các bit 0) **giống nhau**

-Phần P và Q là các bit **khác nhau**

Nhận xét phần prefix chỉ có 8 ký tự, không phải là bội của 64, chương trình sẽ lấp đầy chuỗi bằng phần padding để cho đủ 64 byte, nhưng vẫn phải có phần P hoặc Q tạo chuỗi khác nhau như sau.

$$64(\text{Prefix} + \text{padding}) + 64 (\text{khối P hoặc Q}) + 64 (\text{Khối P hoặc Q}) = 198 \text{ byte}$$

Khi băm cả 2 file

```
eab735bc4d493b3eb071b38ee95bd9b0 out1.bin  
eab735bc4d493b3eb071b38ee95bd9b0 out2.bin
```

Đây là kết quả khi băm cả 2 file output1 và output2, như vậy ta thấy 2 chuỗi đã thỏa yêu cầu là 2 chuỗi khác nhau nhưng băm ra kết quả giống nhau. Chứng minh được MD5 có lỗ hổng.

Câu hỏi 2: Tạo một tệp tiền tố có chính xác 64 byte, chạy lại công cụ xung đột và xem điều gì xảy ra.

1.Cấu hình

```
with open("prefix_exact64.txt", "wb") as f:  
    f.write(b"A" * 64)
```

Khởi tạo Tiền tố có chính xác 64 byte (toàn chữ A)

2.Thực hiện

```
!./md5collgen -p prefix_exact64.txt -o out3.bin out4.bin  
!diff out3.bin out4.bin  
!md5sum out3.bin out4.bin  
!xxd out3.bin | head -n 10  
!xxd out4.bin | head -n 10
```

Chạy lại công cụ tạo xung đột bằng các lệnh và in ra kết quả

3. Kết quả

```

00000000: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000010: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000030: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000040: 77ac 8361 ef24 1d96 4832 b811 345c 6bbe  w..a.$..H2..4\k.
00000050: 4ffc e1fd 2f3a f3f9 3ea6 4411 96b8 3741  O.../::>.D...7A
00000060: 55f3 b3e8 de44 6807 67e6 58f9 f7b7 c95d  U....Dh.g.X....]
00000070: 5c8e 52e9 c075 3f58 ae80 bc69 7427 6e97  \.R..u?X...it'n.
00000080: b23c b946 8cdf 0cdf d50c 06de a442 2965  .<.F.....B)e
00000090: b1b5 2c9b ee1a 874f 7ea9 df4f 5b4d c5c1  ..,....0~..0[M..
000000a0: adab c409 04ca aab6 9795 6b31 3a4d e25e  ....k1:M.^
000000b0: 7199 6e2b 8345 6d9d bec6 db41 2a1d 9608  q.n+.Em....A*...
00000000: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000010: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000030: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000040: 77ac 8361 ef24 1d96 4832 b811 345c 6bbe  w..a.$..H2..4\k.
00000050: 4ffc e17d 2f3a f3f9 3ea6 4411 96b8 3741  O..}/::>.D...7A
00000060: 55f3 b3e8 de44 6807 67e6 58f9 f737 ca5d  U....Dh.g.X..7.]
00000070: 5c8e 52e9 c075 3f58 ae80 bce9 7427 6e97  \.R..u?X....t'n.
00000080: b23c b946 8cdf 0cdf d50c 06de a442 2965  .<.F.....B)e
00000090: b1b5 2c1b ee1a 874f 7ea9 df4f 5b4d c5c1  ..,....0~..0[M..
000000a0: adab c409 04ca aab6 9795 6b31 3acd e15e  ....k1:...^
000000b0: 7199 6e2b 8345 6d9d bec6 dbc1 2a1d 9608  q.n+.Em.....*...

```

Tương tự như ở câu hỏi 1 nhưng thay vì có phần padding để lấp đầy vào 64 byte thì vì bây giờ tiền tố đã có đủ 64 byte rồi nên chỉ cần thêm phần P (hoặc Q) để tạo xung đột thôi.

D. Task2.4

Môi trường thực thi: Google Colab

1. Thực hiện

+Chúng chỉ sẽ được lấy từ “Facebook.com”

+Các bước thực hiện được làm theo ý tưởng của hướng dẫn lab và nằm trong file Task2.4

2. Code Python xác minh chứng chỉ trên RSA

```

import hashlib
S_hex = "05495dc62735c1500e6d11d1ea65ad23ac28e0923433b874d56846746f680f3ac13d4645b044eb8e6b443ba8d11796ec094170e84d3d395fdff96a40e90e2cddcc6f8ee60656287cc194e9cdee8e4825887"
n_hex = "CCF710624FA6B8636FED905256C56D277B7A12568AF1F4F9D6E7E18FB095ABF260411570DB1200FA270AB557385B7D82519371950E6A41945B351BFA7BFA8BC5BE2430FE56EFC4F37097E314F5144DCBA710"
e = 65537

S = int(S_hex, 16)
n = int(n_hex, 16)

# giải mã rsa
em_prime = pow(S, e, n)
em_prime_hex = hex(em_prime)[2:]

h_actual = "69432316407577e1941d455e99eb01398e31d3c75fde437723acb90a08039112"
print(f"Giá trị EM giải mã được (dạng Hex):\n...{em_prime_hex}\n")
if h_actual in em_prime_hex:
    print("Xác minh đúng")
else:
    print("Xác minh sai")

```

+S_hex: chữ ký số được trích xuất từ c0.pem

+n_hex: Modulus trích xuất từ c1.pem

+e: số mũ công khai

+S = int(S_hex, 16)

+n = int(n_hex, 16)

Vì dữ liệu được trích xuất từ OpenSSL ở dạng chuỗi hex nên cần chuyển nó về sang số nguyên để máy tính thực hiện các phép toán số học

```
# giải mã rsa
em_prime = pow(S, e, n)
em_prime_hex = hex(em_prime)[2:]
```

em_prime = pow(S, e, n)

➔ Đây là dòng thực hiện phép toán lũy thừa modulo

➔ **Kết quả (em_prime):** Là một số nguyên cực lớn đại diện cho **Encoded Message (EM)**. Đây là nội dung mà CA đã băm và đóng gói trước khi ký.

em_prime_hex = hex(em_prime)[2:]

➔ Dòng này dùng để chuyển đổi kết quả toán học sang định dạng mà con người có thể đọc và so sánh được. (“[2:]” dùng để loại 2 ký tự đầu 0x để có được chuỗi hex thuần túy)

➔ Chuỗi Hex này để thực hiện bước cuối cùng tìm xem mã băm SHA-256 mà bạn tự tính từ phần thân chứng chỉ (c0_body.bin) có nằm bên trong chuỗi Hex này hay không

-Cuối cùng kiểm tra nếu thấy chuỗi h_actual trong bên trong kết quả giải thì chứng tỏ là nội dung chưa bị thay đổi.

3.Kết quả:

Giá trị EM giải mã được (dạng Hex):

...00042069432316407577e1941d455e99eb01398e31d3c75fde437723acb90a08039112

Xác minh đúng

Thấy chuỗi h_actual nằm trong -> Facebook sở hữu chứng chỉ này và Digicert xác nhận điều đó.

HẾT